GNN Discover Species

This project is a small PyTorch Geometric experiment for learning chemical bonding information in simple H/O combustion-like four-atom systems. The main model learns two related outputs from graph structure and interatomic distances:

- edge-level bond order
- node-level probability that an atom belongs to an existing species

The repository is mostly organized around one final training script, one final plotting script, tutorial/older scripts, repeated training trials, trained checkpoints, and generated molecular visualization figures.

At the time this README was written, the folder contains 511 files, including 30 Python scripts, 22 text logs, 14 PyTorch checkpoints, 221 PNG figures, 218 SVG figures, 1 notebook, 1 VS Code settings file, and 1 PowerPoint deck. Most of the PNG/SVG files are generated outputs rather than hand-written source files.

Quick Start

Run commands from the `gnn_combustion` directory because the scripts use relative paths for checkpoints and output folders.

```
cd
/Users/boyuanyu/Documents/research/GNN/GNN_discover_species/gnn_combustion
```

Train the main model:

```
python ex_bo_4node.py
```

This writes:

```
ex_bo_4node.pth
```

Generate molecule prediction plots from the trained model:

```
python plot_exbo4n.py
```

This reads:

```
ex_bo_4node.pth
```

and writes PNG/SVG figures to:

```
plots/
```

Required Python Packages

The scripts use:

- `torch`
- `torch_geometric`
- `networkx`
- `matplotlib`
- `numpy`

The VS Code settings in `gnn_combustion/.vscode/settings.json` indicate a Conda-based Python environment.

Core Idea

Each training example is a four-node molecular graph containing only hydrogen and oxygen atoms. The four nodes are connected by a complete directed graph: there are 6 undirected atom pairs, represented as 12 directed edges.

Each node has a one-hot feature:

```
H -> [1.0, 0.0]
O -> [0.0, 1.0]
```

Each edge has one feature:

```
edge_attr = interatomic distance
```

The model is trained on synthetic configurations sampled from small H/O species templates such as:

- `H2 + O2`
- `H2 + O + O`

- `O2 + H + H`
- `H2O + O`
- `OH + OH`
- `OH + O + H`
- `H2O2`
- `HO2 + H`
- `H + H + O + O`

The dataset generator randomly permutes atom order so the model cannot rely on fixed node indices.

Training Targets

The bond-order target is computed from a Pauling-style relationship:

```
BO = exp((R0 - R) / b)
```

where:

- `R` is the sampled interatomic distance
- `R0` is the equilibrium distance for the atom pair
- `b = 0.357`

The main script uses:

```
H–H: 0.74 A
O–O: 1.48 A
O–H: 0.97 A
```

Bond order is clamped to a maximum of `2.5`.

The node-existence target is binary:

- `1.0` if the atom participates in at least one bonded pair
- `0.0` if the atom is isolated in that graph

Main Model Architecture

The final model is implemented in `gnn_combustion/ex_bo_4node.py`.

Message Passing Layer

`CombustionConv` extends `torch_geometric.nn.MessagePassing`.

- aggregation: `mean`
- input channels: `2`
- hidden channels: `64`
- distance weighting:

```
message = neighbor_feature * exp(-2.0 * distance)
```

This makes longer-distance edges contribute less strongly during message passing.

Bond Predictor

For each directed edge, the model forms symmetric/commutative edge features:

```
h_u + h_v
h_u * h_v
abs(h_u - h_v)
distance
```

These are concatenated into a 193-dimensional feature vector:

```
64 + 64 + 64 + 1 = 193
```

The bond predictor is an MLP ending in `Softplus`, so predicted bond orders are non-negative.

Existence Predictor

The existence predictor is an MLP applied to each node embedding. It ends in a `Sigmoid`, so the output is a probability between 0 and 1.

Loss

Training minimizes:

```
MSE(predicted bond order, target bond order)
+ BCE(predicted existence, target existence)
```

Main Scripts

`gnn_combustion/ex_bo_4node.py`

Main training script.

What it does:

1. Defines `CombustionConv`.
2. Defines `CombustionGNN`.
3. Generates 1000 synthetic four-node H/O molecular graphs.
4. Trains for 200 epochs with Adam at learning rate `0.001`.
5. Prints total, bond-order, and existence losses every 10 epochs.
6. Runs several hand-coded test cases.
7. Saves model weights to `ex_bo_4node.pth`.

Important implementation details:

- uses a complete directed four-node graph with 12 edges
- uses random atom permutations for each synthetic sample
- includes internal non-bonded distances for molecules such as `H2O`, `H2O2`, and `HO2`
- predicts both edge bond order and node existence probability

`gnn_combustion/plot_exbo4n.py`

Main inference and visualization script.

What it does:

1. Re-defines the same model architecture used by `ex_bo_4node.py`.
2. Loads `ex_bo_4node.pth` from the current working directory.
3. Builds test graphs from explicit 2D coordinates.
4. Computes interatomic distances directly from the coordinates.
5. Runs the GNN to predict bond orders and node existence probabilities.
6. Draws molecule graphs with NetworkX and Matplotlib.
7. Saves both SVG and PNG images into `plots/`.

Visual encoding:

- node label: atom type plus predicted existence probability
- node color: predicted existence probability
- edge width: predicted bond order
- edge label: predicted bond order

The script includes equilibrium and stretched cases, including:

- `H2 + O2`
- `H2O + O`
- `OH + OH`
- `H2O2`
- `HO2 + H`
- `H2 + O + O`
- `O2 + H + H`
- `OH + O + H`
- `H + H + O + O`
- stretched `H2`, `O2`, `OH`, `H2O`, `HO2`, and `H2O2`

Tutorial Scripts

The `gnn_combustion/tutorial` directory contains earlier and simpler versions of the same idea.

`tutorial/test_torch.py`

Minimal PyTorch sanity check. It creates and prints a random tensor.

`tutorial/forward_play.py`

Small forward-pass demo for PyTorch Geometric message passing.

What it demonstrates:

- building a simple `Data` object
- using atom type as a scalar feature
- using edge distances as `edge_attr`
- defining a distance-aware `MessagePassing` layer
- producing untrained bond-order and existence outputs

This is not a training script; it is mainly a conceptual test.

`tutorial/train_small.py`

First small trainable example.

What it does:

- trains on a very simple bonded-vs-dissociated synthetic dataset
- uses scalar atom features
- predicts bond order and existence

- tests one short-distance case and one long-distance case
- saves `combustion_gnn_mini.pth`

This is the simplest working training example in the project.

`tutorial/bo_4node.py`

Older four-node bond-order training script.

What it does:

- uses H/O one-hot node features
- trains only the bond-order predictor
- uses Pauling-style bond-order targets
- tests several H/O species
- saves `combustion_gnn_bo_small.pth`

This version does not train the node-existence branch.

`tutorial/ex_bo_4node.py`

Extended version of `bo_4node.py`.

What it adds:

- node-existence prediction
- binary cross-entropy loss for node existence
- more detailed test cases, including stretched or transition-state-like geometries

It saves `combustion_gnn_bo_small.pth`.

`tutorial/old_ex_bo_train_small.py`

Older experimental version of the extended bond-order/existence model.

Differences from the final script include:

- `aggr='add'` instead of `aggr='mean'`
- a different symmetric edge feature construction
- stronger weighting of bond-order loss
- fewer species templates
- older target constants and bond-order clipping

This file is useful as development history, but the final training script is `gnn_combustion/ex_bo_4node.py`.

Notebook and Reference Figures

`gnn_combustion/loss_plot.ipynb`

Notebook for analyzing training logs and making reference plots.

It does three main things:

1. Reads `ten_tests/<n>/train.txt` files.
2. Plots total loss, bond loss, and existence loss for one or all repeated training runs.
3. Plots Pauling bond order as a function of interatomic distance for H-H, O-H, and O-O bonds.

The notebook writes:

```
pauling_bond_order.png
pauling_bond_order.svg
```

`gnn_combustion/pauling_bond_order.png`

`gnn_combustion/pauling_bond_order.svg`

Reference plots produced by the notebook. They show how the Pauling bond-order formula changes with interatomic distance for H-H, O-H, and O-O pairs.

Repeated Experiment Folders

`gnn_combustion/ten_tests/1` through `gnn_combustion/ten_tests/10`

These folders store 10 repeated runs of the same final training and plotting workflow.

Each run directory contains:

```
ex_bo_4node.py
plot_exbo4n.py
ex_bo_4node.pth
train.txt
```

```
test.txt
plots/
```

The Python scripts in all 10 folders are identical to the main `gnn_combustion/ex_bo_4node.py` and `gnn_combustion/plot_exbo4n.py` scripts. The differences are the trained checkpoint, logs, and generated plots.

File meanings:

- `train.txt` : captured training output from `ex_bo_4node.py`
- `test.txt` : captured inference/plotting output from `plot_exbo4n.py`
- `ex_bo_4node.pth` : trained weights for that run
- `plots/` : PNG/SVG molecule prediction figures for that run

Each `plots/` folder contains 36 generated image files: PNG/SVG pairs for the same set of molecular test cases.

`gnn_combustion/test`

Another saved run of the final workflow. It has the same structure as a single `ten_tests/<n>` folder:

```
ex_bo_4node.py
plot_exbo4n.py
ex_bo_4node.pth
plots/
```

`gnn_combustion/model_for_test`

Archive of earlier trained models, logs, and generated plots.

Important files:

- `combustion_gnn_mini.pth` : checkpoint from the small tutorial model
- `combustion_gnn_bo_small.pth` : checkpoint from the older bond-order model
- `ex_bo_4node.pth` : checkpoint from the extended bond-order/existence model
- `test.txt` : captured training/testing output
- `test_plot.txt` : captured plotting/inference output
- `plots/` : generated molecule visualizations

Other Files

`GNN update_Xu_050626.pptx`

PowerPoint slide deck associated with this GNN species-discovery work.

`gnn_combustion/.vscode/settings.json`

VS Code workspace settings. It configures Python environment/package manager behavior to use the Python extension's Conda support.

`.DS_Store` files

macOS Finder metadata. These are not part of the scientific workflow.

Generated Plot Files

The repository contains many generated PNG/SVG molecular plots. These are outputs of `plot_exbo4n.py`, not source code.

Typical plot filenames include:

- `H2_plus_O2`
- `H2O_plus_O`
- `OH_plus_OH`
- `H2O2`
- `HO2_plus_H`
- `H_plus_H_plus_O_plus_O`
- stretched variants such as `H2_stretched` or `H2O2_stretched`

Each plot shows the model's prediction for a hand-coded molecular geometry.

Recommended Workflow

For a fresh run:

1. Work in `gnn_combustion`.
2. Run `python ex_bo_4node.py`.
3. Inspect the printed training losses and test-case predictions.
4. Run `python plot_exbo4n.py`.
5. Inspect the generated `plots/*.png` or `plots/*.svg` files.
6. Optionally repeat the workflow in a numbered folder under `ten_tests/` to compare training stochasticity across runs.

Notes and Caveats

- The scripts are research prototypes rather than a packaged library.

- Most scripts rely on the current working directory for checkpoints and plot output.
- Random seeds are not fixed, so repeated training runs can differ.
- The model is trained on synthetic four-node H/O systems only; it is not a general molecular GNN.
- The final training and plotting scripts are duplicated in `test/` and `ten_tests/<n>/` for reproducibility of individual runs.
- Checkpoint files (`.pth`) and plot files are generated artifacts.